

Reporte ED - Semana 1

JobNest

Tema: Separacion de servidor publico y privado, hashing de contraseñas y encriptado de datos sensibles en reposo.

Objetivo: implementar medidas iniciales de seguridad para aislar la base de datos, evitar contraseñas en texto plano y proteger datos personales almacenados.

Requisito	Estado
Servidor publico y privado separados	Cumplido
Servidor privado sin acceso directo desde internet	Cumplido con red Docker interna
Hashing de contraseñas con Argon2	Cumplido
Datos sensibles cifrados en reposo	Cumplido
Evidencia por consola / base de datos / sistema	Incluida

1. Definicion de parte publica y privada

En JobNest se identificaron dos zonas principales para cumplir con el entregable de seguridad.

Parte publica

Corresponde al servidor que recibe peticiones de usuarios desde el navegador o una app movil. Incluye el frontend web y el backend Flask expuesto como API / gateway. En la configuracion propuesta el servicio es **jobnest-public** y expone el puerto **5000**.

Parte privada

Corresponde a los componentes que no deben estar expuestos directamente a internet: base de datos SQL Server, datos de usuarios, contraseñas hashadas, datos personales cifrados y logica interna de persistencia. El servicio privado es **jobnest-private-db**.

2. Arquitectura propuesta

La separacion se implementa mediante Docker Compose con dos redes: **frontnet** para el servidor publico y **backnet** para la comunicacion privada con base de datos. La red **backnet** esta configurada con **internal: true**, lo cual la aisla del exterior.

```
flowchart LR
    usuario["Usuario / navegador / app movil"] --> internet["Internet"]
    internet --> publico["Servidor publico: jobnest-public<br/>Flask / API Gateway<br/>Puerto 5000 expuesto"]
    publico --> privada["Red privada Docker: backnet<br/>internal: true"]
    privada --> db["Servidor privado: jobnest-private-db<br/>SQL Server<br/>Puerto 1433 solo expuesto dentro de Docker"]
    atacante["Cliente externo"] -. "No debe conectar directo" .-> db
```

Evidencia 1. Diagrama de arquitectura publico / privado.

En la arquitectura, el usuario solo accede al servicio publico. La base de datos queda en la red privada y solo puede ser alcanzada por el servidor publico dentro de Docker.

```

erick@MacBook-Air-de-Erick JobNest % docker compose -f docker-compose.ed.yml config
name: jobnest
services:
  jobnest-private-db:
    container_name:
    environment:
      ACCEPT_EULA: ""
      MSSQL_PID: Developer
      MSSQL_SA_PASSWORD: E322158b@
    expose:
      - "1433"
    image: mcr.microsoft.com/mssql/server:2022-latest
    networks:
      backnet: null
    platform: linux/amd64
    volumes:
      - type: volume
        source: jobnest_sql_data
        target: /var/opt/mssql
        volume: {}
  jobnest-public:
    build:
      context: /Users/erick/Desktop/JobNest
      dockerfile: Dockerfile.ed
    container_name: jobnest-public
    depends_on:
      jobnest-private-db:
        condition: service_started
        required: true
    environment:
      DATA_ENCRYPTION:
      DB_DRIVER: '{Fr
      DB_ENCRYPT: "no'
      DB_NAME: JobNes
      DB_PASSWORD: E3
      DB_SERVER: jobn
      DB_TIMEOUT: "30"
      DB_TRUST_SERVER_CERTIFICATE: "yes"
      DB_USER: sa
      EMAIL_PASSWORD: ""
      EMAIL_USER: ""
      FLASK_SECRET_KEY: jobnest_dev_secret
      MAIL_PORT: "465"
      MAIL_SERVER: smtp.gmail.com
      MAIL_USE_SSL: "true"
      MAIL_USE_TLS: "false"

```

Evidencia 2. Configuración Docker Compose con servidor público y base privada. Valores sensibles censurados.

```

networks:
  backnet: null
  frontnet: null
ports:
  - mode: ingress
    target: 5000
    published: "5000"
    protocol: tcp
networks:
  backnet:
    name: jobnest_backnet
    internal: true
  frontnet:
    name: jobnest_frontnet
volumes:
  jobnest_sql_data:
    name: jobnest_jobnest_sql_data

```

Evidencia 3. Red backnet configurada como internal: true.

3. Hashing de contraseñas

Se implemento hashing de contraseñas usando **Argon2**. Las contraseñas nuevas no se almacenan en texto plano. El sistema tambien mantiene compatibilidad con hashes anteriores y permite migrarlos cuando el usuario inicia sesion correctamente.

Consulta usada

```
SELECT TOP 5 id, Email, PasswordHash
FROM Usuarios
ORDER BY id DESC;
```

```
sswordHash FROM Usuarios ORDER BY id DESC;"
id      Email      PasswordHash

-----
-----
-----
-----
-----
1004  juan@gmail.com      scrypt:32768:8:1$tKrH8v4DpR9KKENq$9a54d1fc34f1d17020f32f713d320402e602a853e24e409febe69d0a7
9368b372ff84d3f8083be806b0d0fad9463e826a3edcf863e5bbab2f1025f33f76c050a

4  Lalo@gmail.com      scrypt:32768:8:1$8dtanrNliDMJKxzW$b188fc3db2f18e300c6d993ad0a5bfb22a4c1f306d600c5a62333a3e6
0faa03d5d5f65cf1f44ee4d46fd7f7e9bd4cb0d89aee5e377ac34ea5ee499f78f4dd034

3  aron@gmail.com      scrypt:32768:8:1$SPjHSu3P2chcRMoJ$990ec655050b78adedc5d1dedb196304e16ebb9fdb433bf9bd00b2b72
baf3d522e61f98ba85c7c7798e978d11c596f5894614c7d810abf010ef352f85681b65f

2  rafa@gmail.com      scrypt:32768:8:1$XV9Drh2aQJDw8P1W$716c3ffb2d01a2925f757c2b8b6648e339346aba00ecc2348fd855dca
8faf6006367b014927fd3d64f611d67755fdacbb46b41a5e29c5943725bc890a564c7e8

1  erick@gmail.com      scrypt:32768:8:1$nmJUJCzIzoIT0C77$4ffeed365cda3e9f8278dd4ebc05832c6428ca15fd677bfa6fe42b6d8
41c77b82bae98fb37e72f713c661b1705caec34aaaa148779b3c1e5725b4349a279a127

(5 rows affected)
```

Evidencia 4. Usuarios antiguos con hashes previos tipo scrypt; no son texto plano. Comando censurado.

Despues de registrar un usuario nuevo, el campo **PasswordHash** muestra el prefijo **\$argon2id\$**, lo que confirma que la contraseña fue almacenada como hash Argon2.

```
id      Email      PasswordHash

-----
-----
-----
-----
-----
2004  prueba@gmail.com      $argon2id$v=19$m=65536,t=3,p=4$6x1D6L0XI0TQWkspZYwRgg$xbSRrhEH1xvpM6NUVhoI6HX8s0SZcfzzg38UE
0YrB6U
```

Evidencia 5. Usuario nuevo con hash Argon2. Comando censurado.

4. Encriptado de datos sensibles en reposo

Ademas de las contraseñas, se identificaron datos sensibles que requieren proteccion en reposo: telefono del usuario, cuerpo de mensajes, mensajes de solicitudes de servicio e identificadores sensibles de pagos. Estos datos se cifran antes de almacenarse en la base de datos.

Consulta usada

```
SELECT TOP 5 UsuarioId, Telefono
FROM Personas
WHERE Telefono IS NOT NULL
ORDER BY id DESC;
```

El telefono aparece con prefijo **enc:v1:**, por lo que no se guarda como numero legible en la base de datos.

UsuarioId	Telefono
2004	enc:v1:gAAAAABqWmqGUcYVVZkCSMUD-DM9unWRfnjqR0f6z9xQyQotxsmSbhEq3fZe5DJ4ev8_XdVYgSdMsd80gw-cuNT0pJJ028Gu2Q==

Evidencia 6. Telefono cifrado en reposo con prefijo enc:v1:. Comando censurado.

5. Evidencia funcional

Se registro un usuario nuevo y se comprobo que el sistema permite iniciar sesion correctamente. Esto valida que el hashing y el cifrado no rompen el flujo funcional del sistema.



Evidencia 7. Inicio de sesion correcto en JobNest con usuario registrado.

6. Pruebas realizadas

Prueba 1: ejecucion del script **seguridad_semana1.sql** para ajustar columnas sensibles y permitir almacenar valores cifrados.

Prueba 2: consulta a la tabla **Usuarios** para comprobar que **PasswordHash** no contiene contraseñas reales.

Prueba 3: registro de usuario nuevo y verificacion de hash **\$argon2id\$**.

Prueba 4: consulta a la tabla **Personas** para verificar que el telefono queda cifrado con **enc:v1:**.

Prueba 5: revision de Docker Compose para comprobar que la base de datos privada no publica puerto externo y que la red privada esta marcada como interna.

7. Conclusion

Con los cambios realizados, JobNest cumple con el entregable ED - Semana 1. La arquitectura separa la parte publica de la privada usando Docker Compose, el servidor privado no queda expuesto directamente, las contraseñas nuevas se almacenan con Argon2 y los datos sensibles se cifran en reposo.

La implementacion corresponde a la opcion simulada con Docker, aceptada para casos donde no se cuenta con dos VPS reales.